UNIVERSITY OF AUCKLAND
Waipapa Taumata Rau
NEW ZEALAND

SCIENCE

COMPSCI 714
Lectutorial 2: Training and evaluating a DNN with Pytorch

Basics of how to train a DNN with PyTorch and how to evaluate the model's performance.

Instructor: Thomas Lacombe

Learning outcomes

Knowledge:

- Understand several strategies to split the data for training and evaluation a model.
- Recognise several loss functions and when to use them.
- Recognise several evaluation metrics and when to use them.
- Use evaluation metrics to get insights about a model's performance.
- Analyse loss curves to identify underfitting and overfitting.
- Understand the concept of bias and variance.

Practical skills:

- Use PyTorch to define, train and evaluate a model for simple regression and classification tasks

Disclaimer/Acknowledgment:

Some of the following slides are reusing and adapting some of the content created by Dan Klein and Pieter Abbeel for CS188 Intro to AI at UC Berkeley.

All CS188 materials are available at http://ai.berkeley.edu.

Some content is also reused from the CS230 Stanford course slides created by Andrew Ng, available at https://cs230.stanford.edu/syllabus/.

Extra resources

If you want to go further:

- *Deep Learning*, Goodfellow I. et al.
  - Part II Deep Networks, chap. 7 and 8.

- *Hands-on Machine Learning with Scikit-Learn, Keras and Tensorflow*, Géron A.
  - Chapters 4 and 11

PyTorch basics

Empirical and iterative process

Lots of choices to make!

- Evaluation:

Loss function

Evaluation strategy

Evaluation metrics

- NN hyperparameters:

Number of layers

Number of neurons per layer

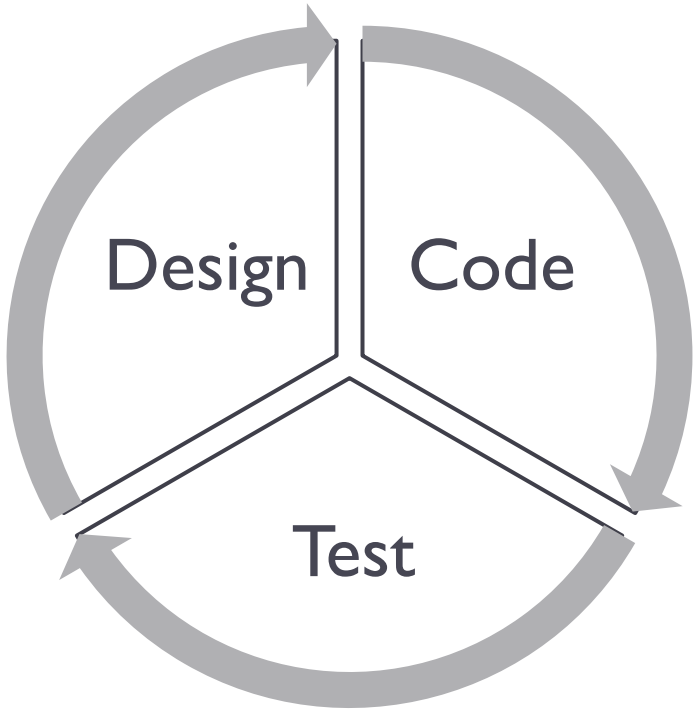
Activation functions

Learning rate

Weight initialisation

Mini-batch size

…



→ Eventually dealt with by AutoML with neural architecture search (NAS)?

https://dl.acm.org/doi/pdf/10.1145/3447582

Evaluation strategy

Validation set is sometimes called development set.

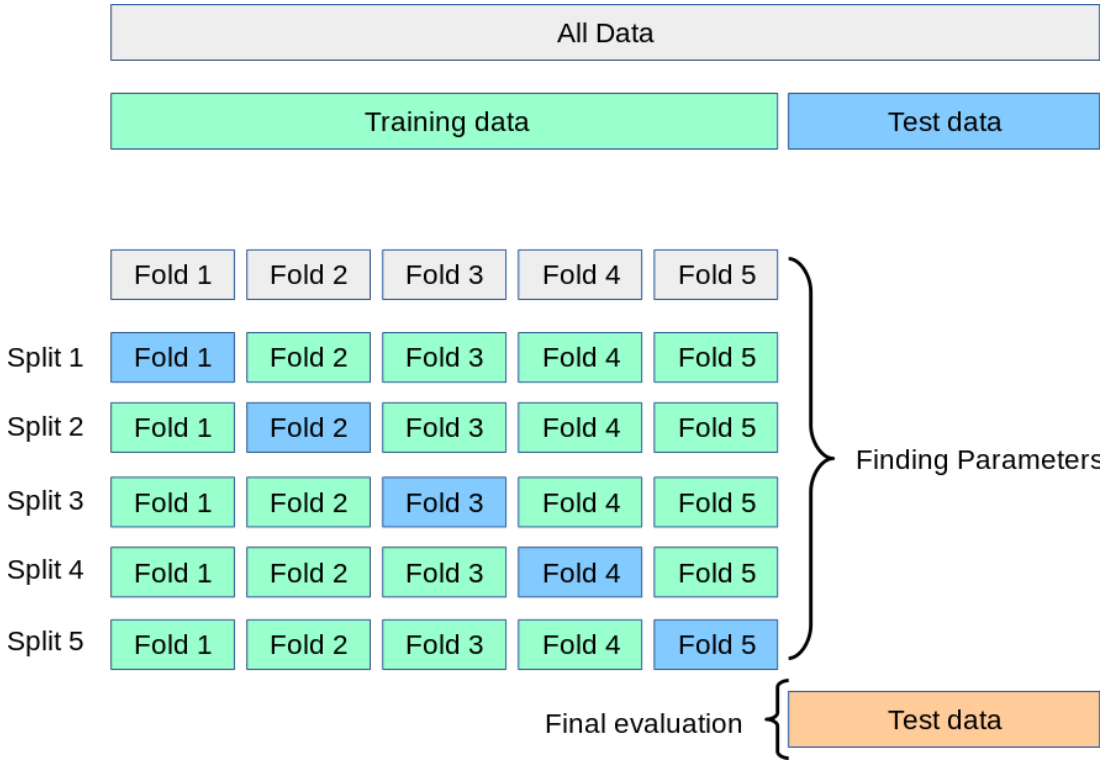
- Train/**validation**/test sets (holdout)

- If small dataset (100, 1000, 10 000 samples)
  - 60%/20%/20%
- If large dataset (> 1 000 000)
  - 98%/1%/1%

- Training set and validation/test set usually need to come from same distribution (but it is ok if it varies a bit when gathering a lot of training data).
- Make sure validation and test sets come from the same distribution.

k-fold cross-validation

- When data set is small, can use cross-validation instead of holdout sets

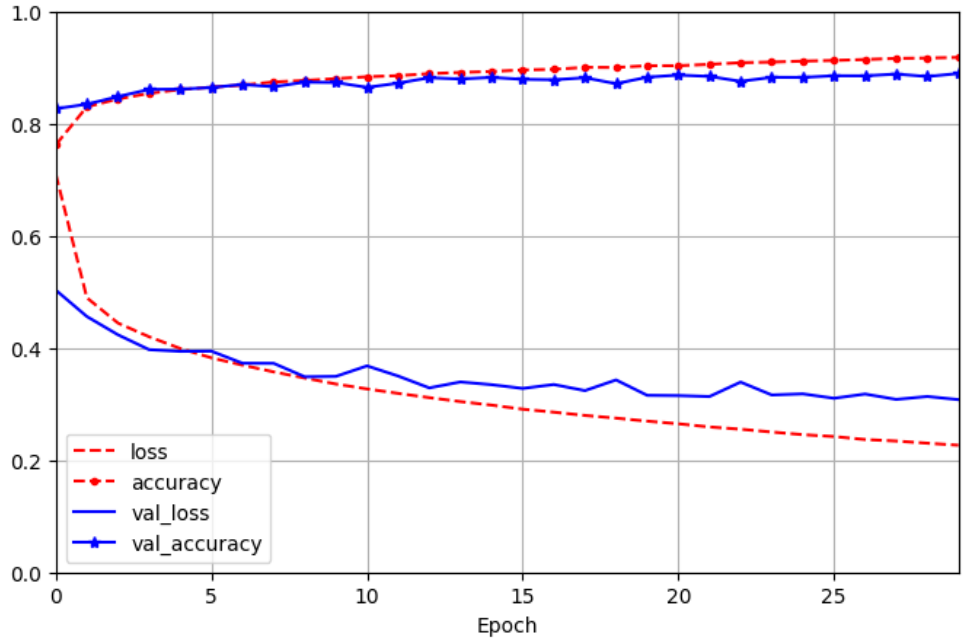


- Leave-one-out cross-validation → special case with k = n, the number of instances in the set

Loss functions

- The loss function is measuring how well the model does (i.e., how do the predicted outputs compare to the expected ones).

- It is usually used as the objective function (usually to be minimised) of the parameter optimisation (i.e., model training).



- There are different loss functions for different tasks and situations.

Loss functions for regression

- Some of the common loss functions for regression are:
  - Mean Squared Error (MSE), a.k.a. L2 loss

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

    - y_i is the predicted value for the i-th sample
    - $\hat{y}_i$ is target value for the i-th sample
    - N is number of instances

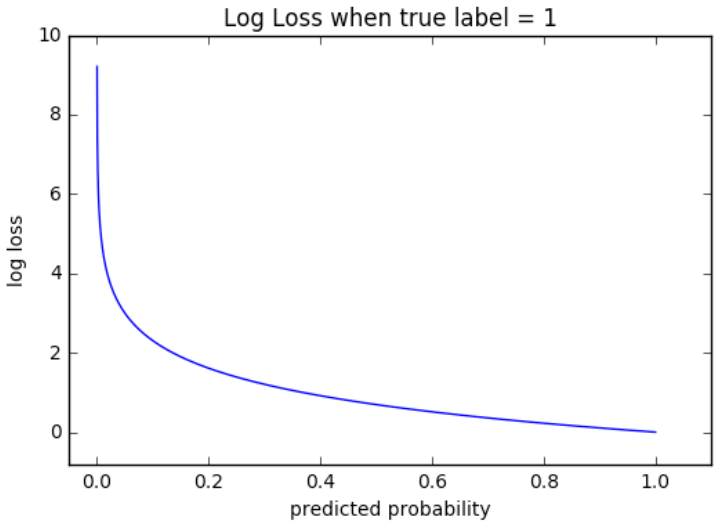
  - Mean Absolute Error (MAE), a.k.a. L1 loss:

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

Notes:

- If you want to get the error in the same unit as the data, you can use RMSE.
- MSE is sensitive to outliers as it squares the difference between the predicted and target value.
- If you have outliers in your data and you do not want to penalise them more than regular data points, prefer using MAE.
- Huber loss might be an interesting option too as it combines MSE and MAE. It penalises more small errors and less large errors.

Loss functions for classification



- Some of the common loss functions for classification are:
  - Binary Cross-Entropy loss, a.k.a. Log loss (binary classification)

$$L(y,p) \;=\; -[y * \log(p) + (1-y) * \log(1-p)]$$

    - y is the true binary label (1 for the positive class, 0 otherwise)
    - p is the probability of the positive class (between 0 and 1) predicted by the model

  - Categorical Cross-Entropy loss (multi-class classification):

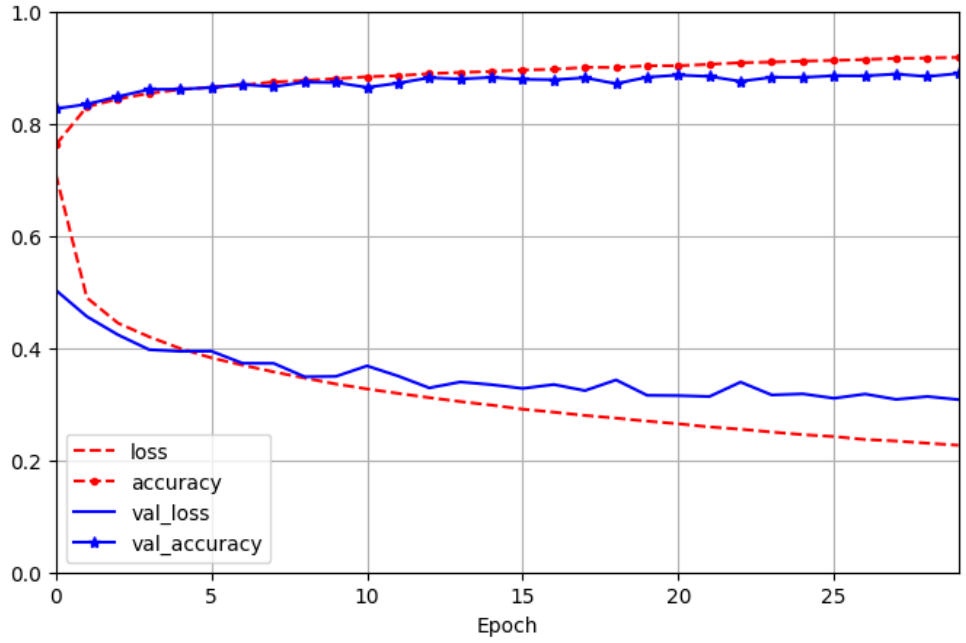
$$L(Y,P) \;=\; -\sum_{i=1}^{i=C} y_i \cdot \log(p_i)$$

    - C is the number of classes
    - y_i is a binary indicator (0 or 1) specifying if label i is the correct classification
    - p_i is the predicted probability for the instance to be of class i

Important:
Cross-entropy loss measure the overall performance, and can therefore be sensitive to class imbalance in the data.

Training a simple DNN for Regression

Evaluation metrics



- The loss can be useful to evaluate if your model learns during training.
- To evaluate the performance of the model on other sets (i.e., dev/valid and test) and on unseen data, we use evaluation metrics.
- Some evaluation metrics are also useful to understand the model's performance more finely.
- The loss and the evaluation metrics are two important aspects of your model evaluation.

Evaluation metrics for classification

- Confusion matrix: tool to analyse the performance of a binary classifier.
  - Each model's prediction is one of 4:
    - True Positive (TP): The model correctly predicted the positive class.
      - Example: identifying a cancer cell as cancerous.
    - True Negative (TN): The model correctly predicted the negative class.
      - Example: identifying a benign cell as benign.
    - False Positive (FP): The model incorrectly predicted the positive class.
      - Example: identifying a benign cell as cancerous.
    - False Negative (FN): The model incorrectly predicted the negative class.
      - Example: identifying a cancerous cell as benign.

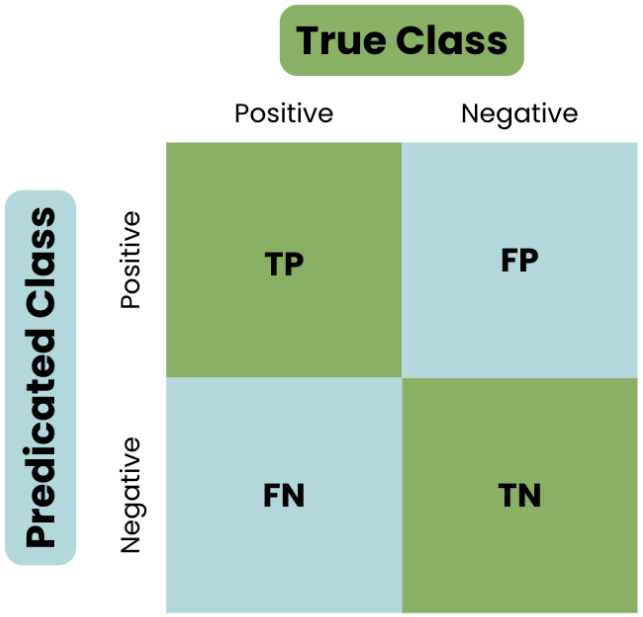


Image source: datacamp

Evaluation metrics for classification

- Several evaluation metrics are derived from the confusion matrix:
  - $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$
  - $Recall = \frac{TP}{TP+FN}$ (sometimes called Sensitivity)
  - $Precision = \frac{TP}{TP+FP}$
  - $Specificity = \frac{TN}{TN+FP}$
  - $F1\ Score = \frac{2 \times Recall \times Precision}{Recall+Precision}$

Accuracy measures the total number of correct classifications over the total number of instances. It is a global measure of performance.

Recall measures the total number of correct positive classifications over the total number of actual positive instances. It is a measure of completeness.
E.g.: It measure how good is your model at identifying all actual cancerous cells. High recall means that the model identify most, if not all cancerous cells. Few actual cancerous cells are missed.

Trade-off

Precision measures the total number of correct positive classifications over the total of predicted positive instances.
E.g.: It measures how good is your model at only predicting actual cancerous cells as cancerous. High precision means that most, if not all cells predicted as cancerous are truly cancerous. Few benign cells are mistaken for cancerous ones.

Image source: datacamp

Confusion matrix example

Predicated Class	True Class: Positive	True Class: Negative
Positive	TP	FP
Negative	FN	TN

Cancerous cells classification example

- 10 000 images in dev/valid set
  - 1000 cancerous
  - 9000 benign
- Confusion matrix on the dev/valid set:
  - TP = 500
  - TN = 8500
  - FP = 500
  - FN = 500

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \frac{9000}{10000} = 0.9\ (90\%)$$

$$Recall = \frac{TP}{TP + FN} = \frac{500}{1000} = 0.5\ (50\%)$$

$$Precision = \frac{TP}{TP + FP} = \frac{500}{1000} = 0.5\ (50\%)$$

$$F1\ Score = \frac{2\ \times Recall\ \times Precision}{Recall + Precision} = \frac{2\ \times 0.5\ \times 0.5}{1} = 0.5$$

Confusion matrix example

		True Class	
		Positive	Negative
Predicated Class	Positive	TP	FP
	Negative	FN	TN

Cancerous cells classification example

- 10 000 images in validation set
  - 1000 cancerous
  - 9000 benign

- Confusion matrix on the validation set:
  - TP = 1000
  - TN = 8000
  - FP = 1000
  - FN = 0

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \frac{9000}{10000} = 0.9\ (90\%)$$

$$Recall = \frac{TP}{TP + FN} = \frac{1000}{1000} = 1\ (100\%)$$

$$Precision = \frac{TP}{TP + FP} = \frac{1000}{2000} = 0.5\ (50\%)$$

$$F1\ Score = \frac{2\ \times Recall\ \times Precision}{Recall + Precision} = \frac{2\ \times 1\ \times 0.5}{1.5} = 0.67$$

Confusion matrix for multi-class

- When you are dealing with a multi-class task, you can either:
  - Build one confusion matrix per class (one against all)
  - Build a multi-class confusion matrix
  - Usually, precision and recall are calculated for each individual class.
  - They can be aggregated using the average recall and precision over all classes.

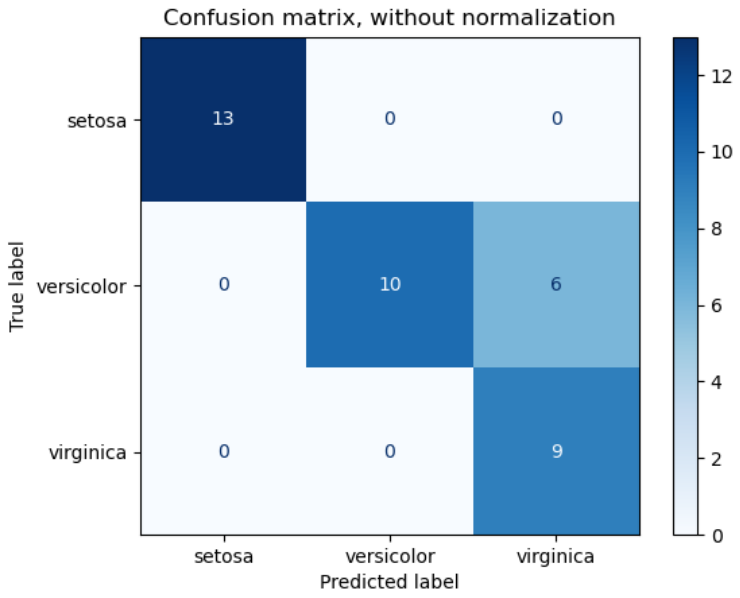


Image source: Scikit-Learn

Evaluation metrics for regression

- You can use MSE / RMSE / MAE directly to evaluate a regression model.

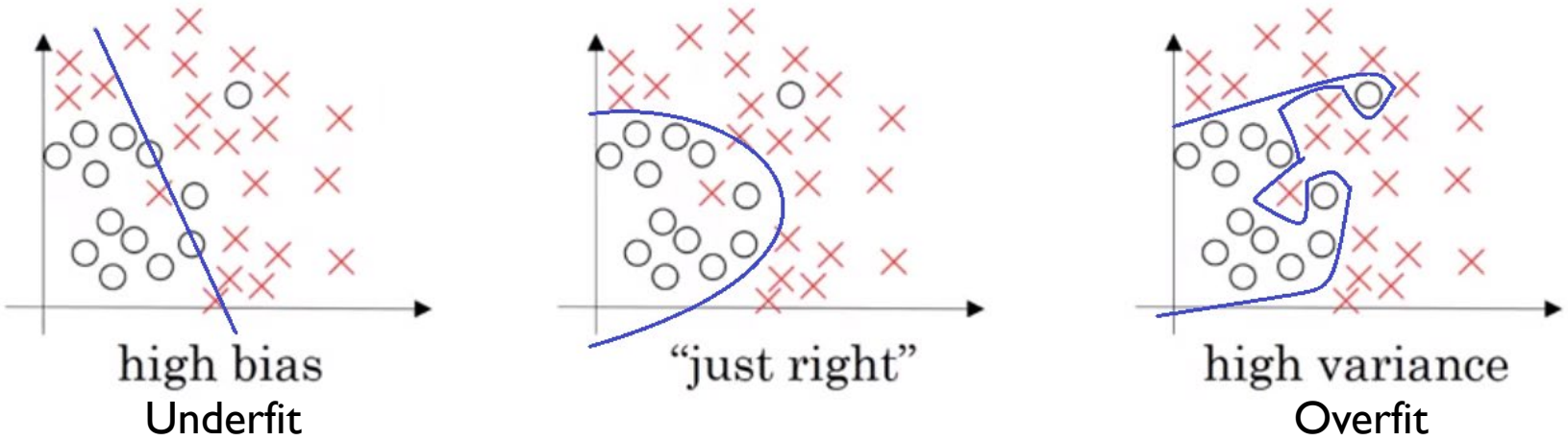
- You can also use R^2:

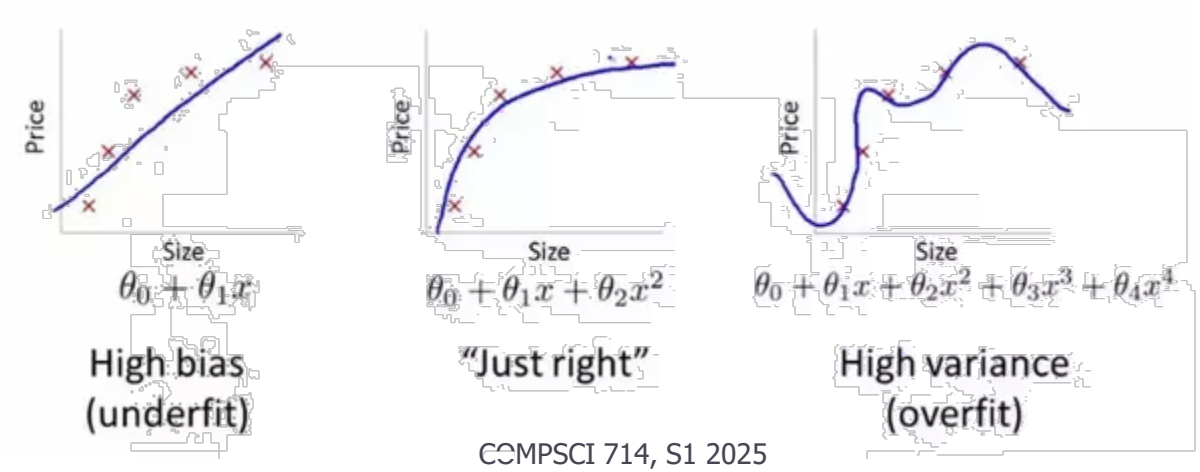
$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

  - y_i is the predicted value for the i-th sample
  - $\hat{y}_i$ is target value for the i-th sample
  - $\bar{y}$ is the mean of y

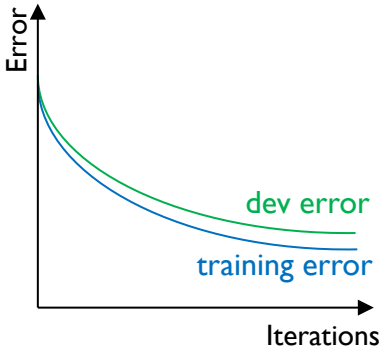
How does you model do?

- Bias vs variance / underfitting vs overfitting



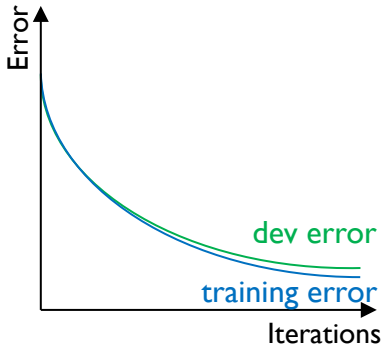


How does you model do?



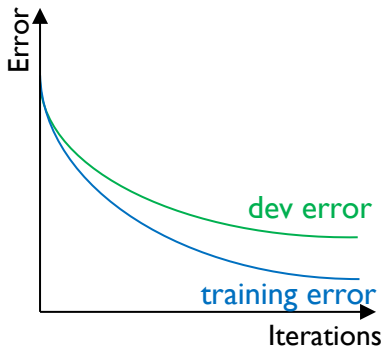
training error = 15%
dev error = 16%

High bias
Underfitting



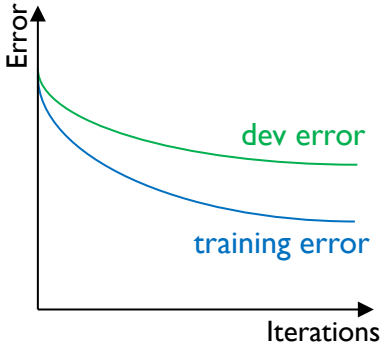
training error = 3%
dev error = 4%

Low bias
Low variance
Good performance



training error = 3%
dev error = 15%

High variance
Overfitting

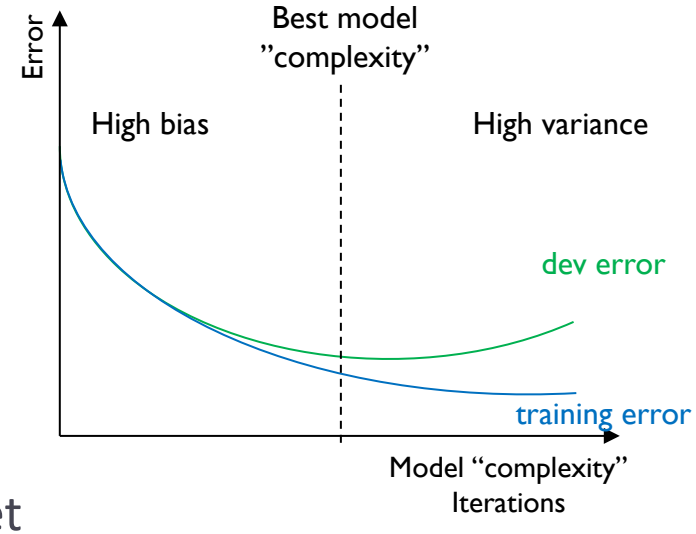


training error = 15%
dev error = 35%

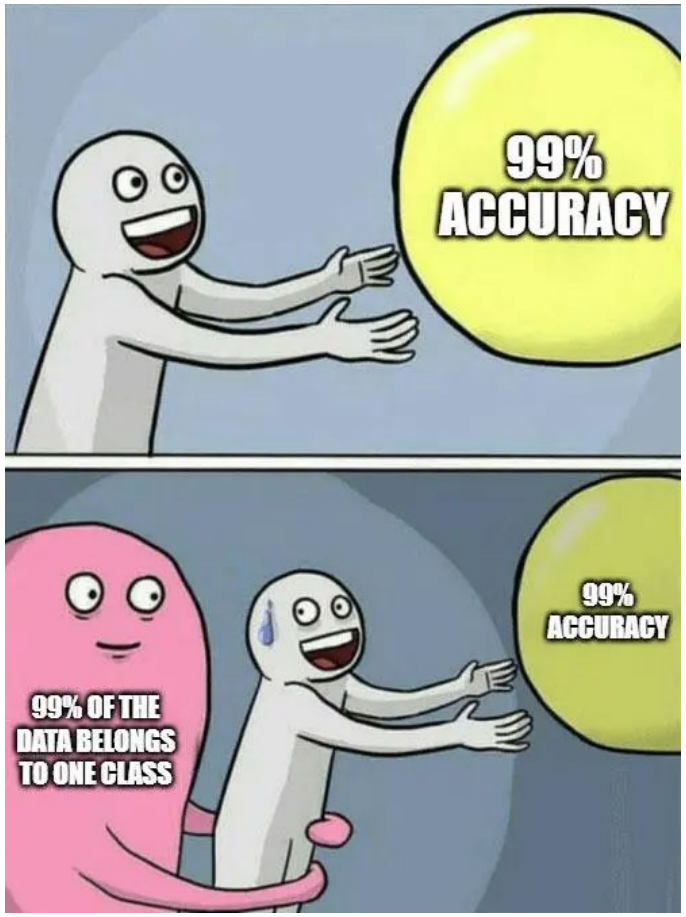
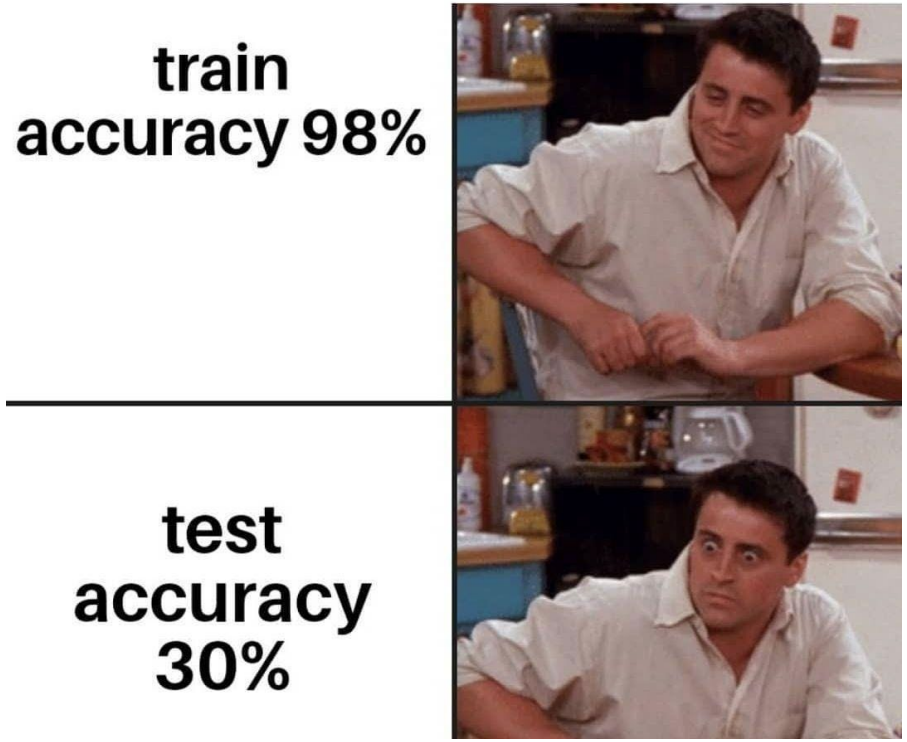
High bias
High variance

The overfitting problem

- Very common problem with Deep Learning: overfitting
  - DNNs have a lot of trainable parameters!
  - Will fit the training data better and better as training goes
  - Up to the point where it might fit the training set perfectly!



- Why is it a problem?
  - If training error = 0 → your model ≡ lookup table on the training set
  - Training data = subset of the all-possible cases (for a task)
  - Model likely useless on the test data and on new data points → dev/test error will be high.
  - Should learn general features, not the exact data points.

Model evaluation

Conclusions